SPRINT 4

NIVEL 1

EJERCICIO 1: Consulta sobre Taula no Optimitzada (Diagnòstic) El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia 12 de març de 2022.
🧩 Tasca:
» 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
» 2. Filtra els resultats per la data indicada i el país "Germany".
» 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

👀 Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).

Consulta sin título | Ejecutar | Guardar | Descargar | Compartir

```sql
SELECT
  t.*,
  c.name,
  c.country
FROM `sprint3-analytics-aherrezuelo.sprint3_silver.transactions_clean` t
JOIN `sprint3-analytics-aherrezuelo.sprint3_silver.companies_clean` c
ON t.company_id = c.company_id
WHERE c.country = 'Germany'
AND t.timestamp >= '2022-03-12'
AND t.timestamp < '2022-03-13';
```

Esta consulta procesará 11.86 MB cuando se ejecute.

He realizado el ejercicio con una consulta que unía las tablas 'transactions_clean' y 'companies_clean' y he filtrado por país y una fecha concreta. Para hacer el filtro de la fecha he usado un rango de fechas en vez de un igual, de esta manera me aseguro que cuente todas las operaciones del día.

EJERCICIO 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

🧩 **Tasca: Pas 1**: Generació de Dades Recents (Mocking Data) Crea una taula intermèdia anomenada sprint3_silver.transactions_recent a partir de la taula sprint3_silver.transactions_clean. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar SELECT * EXCEPT(timestamp). Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions TIMESTAMP_SUB(), CURRENT_TIMESTAMP(), i la combinació CAST(RAND() * 50 AS INT64) per generar els dies de desfasament.

Consulta sin título

Ejecutar | Guardar | Descargar | Compartir

```
CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
SELECT
  * EXCEPT(timestamp),
  TIMESTAMP_SUB(
    CURRENT_TIMESTAMP(),
    INTERVAL CAST(RAND() * 50 AS INT64) DAY
  ) AS timestamp
FROM `sprint3_silver.transactions_clean`;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Detalles de la ejecución | Gráfico de ejecución

Esta declaración creó una nueva tabla con el nombre transactions_recent.

Ir a la tabla

Nombre del campo	Tipo	Modo
transaction_id	STRING	NULLABLE
card_id	STRING	NULLABLE
company_id	STRING	NULLABLE
amount	FLOAT	NULLABLE
declined	INTEGER	NULLABLE
product_ids	INTEGER	REPEATED
user_id	INTEGER	NULLABLE
lat	FLOAT	NULLABLE
longitude	FLOAT	NULLABLE
timestamp	TIMESTAMP	NULLABLE

Fila	transaction_id	card_id	company...	amount	decli...	product_ids	user...	lat	longitude	timestamp
1	F3E47D02-CE43-4B87-8C3F-5B...	CcU-4856	b-2286	322.53	0	40	1	40.70754791...	-74.0159698...	2026-04-17 06:43:57.712271 UTC
						47				
						56				
2	8FA78D29-6180-439E-ADA1-F7...	CcU-4856	b-2534	298.56	0	22	1	40.70526699...	-74.0026638...	2026-04-14 06:43:57.712271 UTC
						55				
						95				
						53				
3	6ED484BC-2DBB-47CF-B77E-12...	CcU-4856	b-2242	233.52	0	85	1	40.71884074...	-74.0131030...	2026-04-09 06:43:57.712271 UTC
						56				
						55				
4	1EDDC7AC-D3CA-440B-A048-3...	CcU-4856	b-2426	254.46	0	94	1	40.70699307...	-74.0054961...	2026-04-15 06:43:57.712271 UTC
						96				

Tabla transactions_clean

Fila	transaction_id	card_id	compan...	timestamp	amount	declined	product_ids	user_id	lat	longitude
1	6ED484BC-2DBB-47CF-B77E-12...	CcU-4856	b-2242	2015-05-08 18:09:41 UTC	233.52	0	85	1	40.7188407...	-74.0131030...
							56			
							55			
2	E505F900-E50F-480B-8A78-7C...	CcU-4856	b-2578	2019-07-07 03:43:38 UTC	124.04	0	50	1	40.7207374...	-74.0146604...
3	7E438CE2-B3E0-4C8C-9EA7-0D...	CcU-4856	b-2530	2018-06-15 18:33:31 UTC	554.76	0	67	1	40.7139257...	-73.9981356...
							20			
							23			
							21			
4	78F14A51-F321-4CA9-AB42-02...	CcU-4856	b-2282	2018-10-24 09:46:13 UTC	265.14	0	17	1	40.7036804...	-74.0080139...
							66			
							3			
5	70C3BE45-B922-470B-B7CF-EC...	CcU-4856	b-2494	2018-02-12 18:09:59 UTC	442.23	0	74	1	40.7183569...	-74.0109240...

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering) Ara, crea la teva taula defi nitiva sprint3_gold.fact_transactions_optimized a partir de les dades recents que acabes de generar a transactions_recent. Has de construir la sentència DDL per confi gurar la taula amb les següents estratègies físiques d'emmagatzematge:

» Particionament (Partitioning): Divideix la taula per la data del camp DATE(timestamp). Això permetrà que les consultes que fi ltrin per dia (WHERE date = ...) només llegeixin la partició necessària.
» Clustering: Ordena les dades dins de cada partició per business_id. Això accelerarà dràsticament els fi ltres per companyia i els encreuaments (JOINs) amb la dimensió d'empreses.

Consulta sin título

Ejecutar | Guardar | Descargar | Compar

```
CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY company_id AS

SELECT *
FROM `sprint3_silver.transactions_recent`;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo | Resultados | Detalles de la ejecución | Gráfico de ejecución

Esta declaración creó una nueva tabla con el nombre fact_transactions_optimized.

Fila	transaction_id	card_id	company_id	amount	declined	product_ids	user...	lat	longitude	timestamp
1	770FE166-59C1-4C6A-9C70-FC...	CcU-3568	b-2222	376.87	0	57 75 39	185	55.6553351...	-3.15072392...	2026-04-18 06:43:57.712271 UTC
2	D0DB6EB4-7C4D-4117-9C78-7B...	CcS-9174	b-2222	106.95	0	69 11	4593	50.8949523...	10.3494715...	2026-04-18 06:43:57.712271 UTC
3	4127E947-E9E7-4937-827A-2FE...	CcS-9011	b-2222	490.62	0	13 15 71 26	4430	41.1520750...	-8.63740729...	2026-04-18 06:43:57.712271 UTC

La partición reduce la cantidad de bytes que se leen al filtrar solo las fechas que necesitamos. El clustering optimiza la organización de los datos, haciendo que los filtros y las uniones sean más eficientes. Por lo que, se reduce el volumen de datos y por lo tanto el coste de la consulta.

EJERCICIO 3: Prova del Cotó (Benchmark):

🧩 Tasca: L'objectiu és clar i directe: aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional. Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i fi ltri les dades dels últims 30 dies. ⚠️ NO executis les consultes encara. En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els Bytes processats de manera gratuïta. Fes la comparativa seguint aquests dos passos:

» **Pas 1** (Taula no optimitzada): Apunta la teva consulta a la taula sprint3_silver.transactions_recent. Observa el validador i fes una captura de pantalla dels bytes que processaria.
» **Pas 2** (Taula optimitzada): Sense canviar res de la lògica de la teva consulta, modifi ca només el FROM perquè ara apunti a la taula particionada sprint3_gold.fact_transactions_optimized. Observa com canvia el validador i fes la segona captura.

Tabla no optimizada:

```sql
SELECT *
FROM `sprint3_silver.transactions_recent`
WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```

✅ Esta consulta procesará 11.85 MB cuando se ejecute.

Tabla optimizada:

```sql
SELECT *
FROM `sprint3_gold.fact_transactions_optimized`
WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```

✅ Esta consulta procesará 7.24 MB cuando se ejecute.

Comparando las dos consultas comparando las dos tablas (recent y optimized) se ve que con la tabla optimizada reduce el gasto en 4.61 MB. El clustering por company_id hace que optimice mejor ya que accede solo a la parte que le decimos en el filtro.
La partición nos da acceso a la ''carpeta'' fecha y el clustering hace que vayamos directo al ''documento'' company_id dentro de esa carpeta.

Diferencia ⇒ 11.85 MB - 7.24 MB = 4.61 MB
Ahorro ⇒ (4.61 / 11.85)*100 = 39%

Gracias al particionado y al clustering se ha ahorrado un coste de aproximadamente 39%

EJERCICIO 4: Smart Caching (Vistes Materialitzades) Escenari:

El Director General té un quadre de comandament que mostra les "Vendes Totals per Dia". Ell (i 50 managers més) refresquen aquest gràfi c constantment.

Problema: Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de fi les. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució: Utilitzar una Vista Materialitzada. Aquesta tecnologia guarda el resultat del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

🧩 Tasca:
» Crea una Vista Materialitzada anomenada sprint3_gold.mv_daily_sales que mostri les Vendes Totals per Dia.

```sql
CREATE OR REPLACE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS
SELECT
  DATE(timestamp) AS date,
  SUM(amount) AS total_sales
FROM `sprint3_gold.fact_transactions_optimized`
WHERE declined = 0
GROUP BY date;
```

Se completó la consulta

```sql
SELECT *
FROM `sprint3_gold.mv_daily_sales`;
```

Se completó la consulta

Fila	date	total_sales
1	2026-04-12	527649.9400000...
2	2026-03-18	507082.4799999...
3	2026-03-27	517890.5000000...
4	2026-03-12	491522.3100000...
5	2026-04-08	517580.0000000...
6	2026-04-06	514017.1100000...
7	2026-04-22	513283.8900000...
8	2026-03-30	524523.4499999...

Bytes procesados	Undefined parameter - BYTES_CONTAINER (resultados almacenados en caché)
Bytes facturados	0 B

Con la creación de vistas materializadas conseguimos que las consultas reutilicen resultados precompuestos, reduciendo el coste a 0 bytes procesados, al no tener que calcularlo cada vez que se hace, como en una consulta regular, mejoramos significativamente el rendimiento, el tiempo de consulta y dinero.

Nivell 2: SQL Analític Avançat

EJERCICIO 1 : Perfi lat de Clients VIP (Mètriques Agregades amb CTEs)

🧩 Tasca:
» 1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:
○ La Despesa Total (SUM).
○ La Quantitat de Transaccions (COUNT).
○ El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
○ La Compra Màxima (MAX).
○ Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» Creua la CTE amb users_combined per obtenir les dades personals. Requisits de Sortida:
○ Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
○ Ordenat per total_gastat descendent.

```sql
WITH VIP_Stats AS (
  SELECT
    user_id,
    SUM(amount) AS total_gastado,
    COUNT(*) AS num_compras,
    ROUND(AVG(amount), 2) AS tiquet_medio,
    MAX(amount) AS max_compra
  FROM `sprint3_gold.fact_transactions_optimized`
  GROUP BY user_id
  HAVING total_gastado > 500
)
SELECT
  v.user_id,
  u.name,
  u.surname,
  u.email,
  v.num_compras,
  v.tiquet_medio,
  v.max_compra,
  v.total_gastado
FROM VIP_Stats v
JOIN `sprint3_silver.users_combined` u
ON v.user_id = u.user_id
ORDER BY v.total_gastado DESC;
```

✅ Se completó la consulta

Fila	user_id	name	surname	email	num_compras	tiquet_medio	max_compra	total_gastado
1	289	Dxwgi	Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.53999999...
2	318	Bnyr	Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.5
3	455	Fwfll	Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.59	724.44	34491.27000000...
4	417	Fqatil	Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	78	410.46	675.78	32016.10999999...
5	185	Molly	Gilliam	donec@outlook.couk	110	275.75	659.31	30333.02999999...
6	285	Brcxprnj	Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva	Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05000000...
8	454	Sfzzoh	Xgvfridxs	sfzzoh.xgvfridxs@example.com	81	300.67	604.9	24353.91000000...
9	349	Yxxcjk	Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47

La estructura de la consulta es la siguiente:

- CTE : tabla temporal con los cálculos y agrupaciones.
- En la siguiente parte mostramos aquellos datos que hemos calculado en la tabla temporal.
- Por último, unimos la tabla temporal se une con la tabla 'user_combined' para que podamos mostrar también el id y nombre completo.

Ejercicio 2: Anàlisi de Tendències (Window Functions sobre Vistes)

📈 Estratègia d'Optimització: Per a aquest càlcul NO has d'anar a la taula de fets original ni recalcular sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada sprint3_gold.mv_daily_sales que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

🧩 Tasca: Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- Data: La data de venda.
- Vendes_Avui: El total venut aquell dia.
- Vendes_Ahir: El total venut el dia anterior (usant funcions de fi nestra).
- Diff_Percentual: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

```sql
SELECT
  date,
  total_sales AS ventas_hoy,

  LAG(total_sales) OVER (ORDER BY date) AS ventas_ayer,

  ROUND(
    (total_sales - LAG(total_sales) OVER (ORDER BY date))
    / LAG(total_sales) OVER (ORDER BY date) * 100,
    2
  ) AS dif_percentual

FROM `sprint3_gold.mv_daily_sales`
ORDER BY date;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Fila	date	ventas_hoy	ventas_ayer	dif_percentual
1	2026-03-06	272885.0100000...	*null*	*null*
2	2026-03-07	530328.4600000...	272885.0100000...	94.34
3	2026-03-08	528534.3300000...	530328.4600000...	-0.34
4	2026-03-09	523655.18	528534.3300000...	-0.92
5	2026-03-10	516930.3400000...	523655.18	-1.28
6	2026-03-11	530991.1300000...	516930.3400000...	2.72
7	2026-03-12	492773.4900000...	530991.1300000...	-7.2
8	2026-03-13	506460.5000000...	492773.4900000...	2.78
9	2026-03-14	535899.2000000...	506460.5000000...	5.81
10	2026-03-15	540707.5100000...	535899.2000000...	0.9
11	2026-03-16	515347.7800000...	540707.5100000...	-4.69
12	2026-03-17	526702.7800000...	515347.7800000...	2.2

Con esta consulta lo que se pretende es comparar las ventas del día con el día anterior y ver si hubo un incremento o decremento (el resultado del primer día sale con resultado null porque el día anterior no hay datos).

Con el comando LAG obtenemos la venta del día anterior para después poder calcular ese porcentaje de variación de venta.
Está ordenado por fecha para que se pueda entender bien los resultados.

EJERCICIO 3: Totals Acumulats (Running Totals sobre Vistes)

🧩 Tasca: Utilitzant la vista materialitzada mv_daily_sales (per no recalcular agregacions base), genera un informe amb tres columnes:

» 1.Data.
» 2.Vendes del Dia: Arrodonit a 2 decimals.
» 3.Vendes Acumulades YTD: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat fi nal ha d'estar arrodonit a 2 decimals.

💡 Pista Tècnica: Per aconseguir el reinici anual, necessites dividir la fi nestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

La estructura del código es la siguiente:

- Se muestra la fecha, la suma total de ventas redondeadas a dos decimales.
- La suma progresiva desde la primera fecha del año hasta la fecha actual. Reiniciando el cálculo cada año mediante PARTITION BY. Esto permite analizar la evolución progresiva de ingresos sin necesidad de múltiples consultas o cálculos adicionales.
- Todo ordenado por fecha en ascendente para que la fecha más reciente se vaya colocando al final del registro de datos.

```sql
SELECT
  date,

  ROUND(total_sales, 2) AS ventas_dia,

  ROUND(
    SUM(total_sales) OVER (
      PARTITION BY EXTRACT(YEAR FROM date)
      ORDER BY date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ),
    2
  ) AS ventas_acumuladas_ytd

FROM `sprint3_gold.mv_daily_sales`

ORDER BY date;
```

Fila	date	ventas_dia	ventas_acumulad...
1	2026-03-06	272885.01	272885.01
2	2026-03-07	530328.46	803213.47
3	2026-03-08	528534.33	1331747.8
4	2026-03-09	523655.18	1855402.98
5	2026-03-10	516930.34	2372333.32
6	2026-03-11	530991.13	2903324.45
7	2026-03-12	492773.49	3396097.94
8	2026-03-13	506460.5	3902558.44
9	2026-03-14	535899.2	4438457.64
10	2026-03-15	540707.51	4979165.15
11	2026-03-16	515347.78	5494512.93

EJERCICIO 4: Fidelització i Valor del Client (Filtratge Avançat) .

🧩 Tasca: Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

» 1.Dades d'usuari (user_id, nom_complet, email).
» 2.La data i l'import exacte de la 3a compra.
» 3.Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

💡 Pista Tècnica:
» Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres.
» Has d'utilitzar la clàusula QUALIFY per fi ltrar.
» Repte d'Optimització: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta fi ltrar les fi les necessàries abans de realitzar l'agregació final.

```sql
WITH primeras_compras AS (
  SELECT
    user_id,
    timestamp,
    amount,

    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY timestamp, amount
    ) AS num_compra

  FROM `sprint3_gold.fact_transactions_optimized`

  QUALIFY num_compra <= 3
),

media_por_usuario AS (
  SELECT
    user_id,
    ROUND(AVG(amount), 2) AS media_3_primeras
  FROM primeras_compras
  GROUP BY user_id
)

SELECT
  p.user_id,
  u.name,
  u.surname,
  u.email,

  p.timestamp AS fecha_3a_compra,
  p.amount AS importe_3a_compra,

  m.media_3_primeras

FROM primeras_compras p

JOIN media_por_usuario m
ON p.user_id = m.user_id

JOIN `sprint3_silver.users_combined` u
ON p.user_id = u.user_id

WHERE p.num_compra = 3;
```

Se completó la consulta

Fila	user_id	name	surname	email	fecha_3a_compra	importe_3a_com...	media_3_primeras
1	44	Aquila	Haley	tempus.risus@outlook.edu	2026-03-10 06:43:57.712271 UTC	269.81	234.76
2	85	Dawn	Murray	ante.dictum.mi@outlook.couk	2026-03-08 06:43:57.712271 UTC	26.66	180.34
3	259	Slade	Downs	nunc@protonmail.net	2026-03-09 06:43:57.712271 UTC	238.41	145.28
4	317	Qikcvc	Nvfcifqdh	qikcvc.nvfcifqdh@example.com	2026-03-07 06:43:57.712271 UTC	467.0	436.26
5	555	Fjhizg	Edujjfsp	fjhizg.edujjfsp@example.com	2026-03-17 06:43:57.712271 UTC	72.79	131.95
6	593	Vljbkf	Rsnfsmzk	vljbkf.rsnfsmzk@example.com	2026-03-22 06:43:57.712271 UTC	85.26	281.46
7	650	Hfuvap	Yuxlusrj	hfuvap.yuxlusrj@example.com	2026-03-09 06:43:57.712271 UTC	374.1	354.59
8	823	Zbvxri	Qtepfqyl	zbvxri.qtepfqyl@example.com	2026-03-19 06:43:57.712271 UTC	91.89	292.86

He utilizado ROW_NUMBER para ordenar las compras por usuario y QUALIFY para filtrar directamente la tercera transacción, optimizando la consulta. Además, calcula la media de las tres primeras compras limitando previamente los datos, reduciendo el coste computacional.

Nivel 3

Ejercicio 1: Desanidament i Aplanament de Dades (Unnesting)

🧩Tasca: Crea la taula dim_transactions_flat desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

⚙️ Passos Tècnics:
» 1. Utilitza CROSS JOIN UNNEST(product_ids) per transformar l'Array en fi les individuals.
» 2. Realitza un JOIN amb la taula products per obtenir el nom (name) i preu (price) de cada ítem. 📍 Nota: Ves amb compte amb els tipus de dades en fer el JOIN (Array de INT64 vs Product_ID STRING).
» 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

```sql
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
SELECT
  t.transaction_id,
  t.timestamp,
  t.amount AS total_ticket,

  product_id,

  p.name,
  p.price

FROM `sprint3_silver.transactions_clean` t

CROSS JOIN UNNEST(t.product_ids) AS product_id

JOIN `sprint3_silver.products_clean` p
ON product_id = p.product_id

WHERE t.declined = 0;
```

Se completó la consulta

1	B359852A-7F87-422E-B049-51...	2017-07-03 21:47:57 UTC	417.72	14	Direwolf	147.53
2	8831A7C6-040B-41DF-BBE5-6E...	2023-10-19 04:30:08 UTC	445.38	14	Direwolf	147.53
3	5975B711-ACE5-4ACE-A34C-90...	2020-11-30 15:29:23 UTC	703.68	14	Direwolf	147.53
4	462E6786-4A69-46F4-8992-E2...	2015-09-17 18:59:20 UTC	147.53	14	Direwolf	147.53
5	5D007E86-E8F8-4009-B206-7E0...	2019-09-05 17:16:11 UTC	472.71	14	Direwolf	147.53
6	B5382D88-33E1-4085-8951-EE...	2018-02-01 02:31:02 UTC	319.1	14	Direwolf	147.53
7	62F19371-7B44-4C30-AA8A-F9...	2024-06-30 06:23:22 UTC	423.77	14	Direwolf	147.53
8	B910CED7-F256-4943-92ED-D8...	2016-02-03 05:43:10 UTC	340.13	14	Direwolf	147.53
9	873E7298-B7E7-4832-8097-16C...	2022-07-29 11:00:07 UTC	189.38	14	Direwolf	147.53
10	023A0F91-F62E-49F3-84DD-27...	2021-01-07 09:25:46 UTC	220.84	14	Direwolf	147.53
11	3A26E0D5-882E-43C9-BC92-B4...	2023-03-01 05:42:01 UTC	314.68	14	Direwolf	147.53
12	41377DE7-B704-4475-868E-823...	2023-02-24 22:46:11 UTC	437.97	14	Direwolf	147.53

He creado la tabla 'dim_transactions_fla't con el uso de CROSS JOIN UNNEST para desnormalizar el array de productos en filas separadas y un JOIN con la tabla 'products_clean' para añadir los datos de nombre y precio.

EJERCICIO 2: El Rànquing de Vendes (Agregació Simple)

🧩 Tasca: Genera el Top 5 de productes més venuts en la història de la companyia.
🛠️ Mètode: Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).
» En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les fi les.

👉 Entrega:
» El codi SQL que genera el rànquing.
» Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

```sql
SELECT
  product_id,
  product_name,
  COUNT(*) AS total_vendidos

FROM `sprint3_gold.dim_transactions_flat`

GROUP BY product_id, product_name

ORDER BY total_vendidos DESC

LIMIT 5;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Fila	product_id	product_name	total_vendidos
1	52	riverlands the duel	2654
2	29	Tully maester Tarly	2635
3	21	duel Direwolf	2609
4	16	the duel warden	2608
5	66	mustafar jinn	2601

He generado el ranking de los 5 productos más vendidos utilizando COUNT sobre la tabla desnormalizada 'dim_transactions_flat', agrupando por nombre de producto y ordenando de mayor a menor número de ventas.

EJERCICIO 3: Automatització del Pipeline i Visualització

🧩 Tasca: Has d'evolucionar la teva taula dim_transactions_flat per incloure el càlcul d'impostos i automatitzar la seva regeneració. ⚙️ Passos Tècnics:
» 1. User Defi ned Functions (UDF): Crea una funció SQL persistent anomenada calculate_tax(amount) que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
» 2. Integració i Orquestració:
○ Modifi ca el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció calculate_tax i generi una columna nova: product_price_tax_inc.
○ Confi gura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
» 3. Visualització (BI): Connecta Looker Studio a la teva taula dim_transactions_flat i crea un Dashboard: "Monitor de Rendiment de Vendes".

Con la función UDF se puede calcular el impuesto y se ha integrado en la tabla final, automatizando su ejecución mediante una consulta programada. El dashboard de Looker Studio permite visualizar KPIs, evolución temporal y rendimiento de productos.

```sql
CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)
RETURNS FLOAT64
AS (
  amount * 1.21
);
```

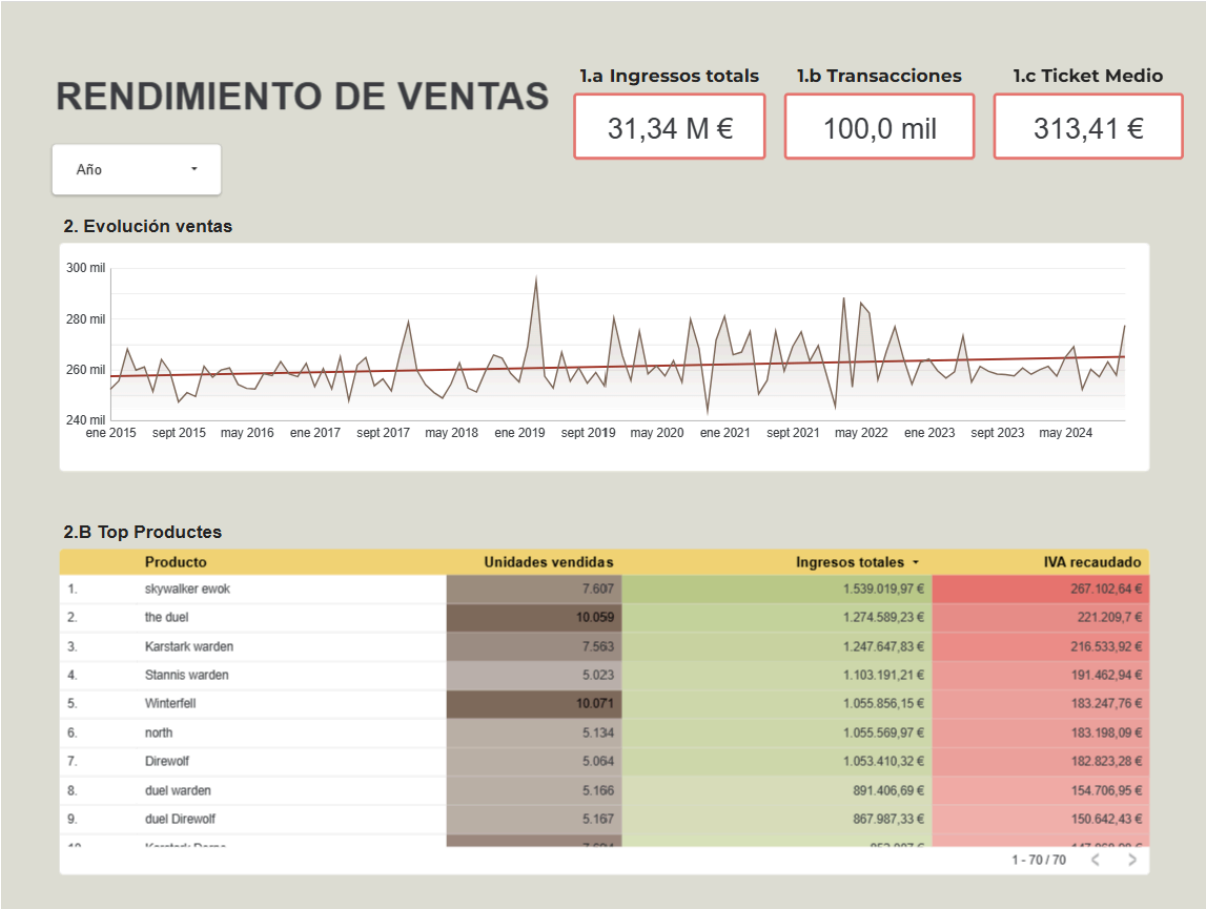
Se completó la consulta

```sql
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS

SELECT
  t.transaction_id,
  t.timestamp,
  t.amount AS total_ticket,

  product_id,

  p.name AS product_name,
  p.price AS product_price,

  `sprint3_gold.calculate_tax`(p.price) AS product_price_tax_inc

FROM `sprint3_silver.transactions_clean` t

CROSS JOIN UNNEST(t.product_ids) AS product_id

JOIN `sprint3_silver.products_clean` p
ON product_id = p.product_id;
```

Se completó la consulta

Informe Locker

enlace : https://datastudio.google.com/s/pb4qkGUUv6M



	Producto	Unidades vendidas	Ingresos totales	IVA recaudado
1.	skywalker ewok	7.607	1.539.019,97 €	267.102,64 €
2.	the duel	10.059	1.274.589,23 €	221.209,7 €
3.	Karstark warden	7.563	1.247.647,83 €	216.533,92 €
4.	Stannis warden	5.023	1.103.191,21 €	191.462,94 €
5.	Winterfell	10.071	1.055.856,15 €	183.247,76 €
6.	north	5.134	1.055.569,97 €	183.198,09 €
7.	Direwolf	5.064	1.053.410,32 €	182.823,28 €
8.	duel warden	5.166	891.406,69 €	154.706,95 €
9.	duel Direwolf	5.167	867.987,33 €	150.642,43 €